

6. RECURSION (BASIC LEVEL – MUST MASTER)

You must clearly separate **3 things** every time:

1. **Base case** (when recursion stops)
2. **Recursive call** (problem becomes smaller)
3. **What happens AFTER the call returns**

If you mix these → recursion breaks.

A. Absolute Basics (DO NOT SKIP)

Problem 6.1 – Countdown

Task:

Print numbers from n to 1 using recursion.

Rules:

- No loops
- One recursive call

Questions you must answer:

- What is the base case?
- What happens if base case is removed?

☐ *This teaches stopping condition.*

Problem 6.2 – Count Up

Task:

Print numbers from 1 to n using recursion.

☐ Hint: Printing position matters.

Question:

Why is the print statement placed **after** the recursive call?

☐ *Understanding call stack order.*

B. Return-Based Recursion (IMPORTANT)

Problem 6.3 – Sum of First N Numbers

Task:

Return sum of numbers from 1 to n.

`sum(n) = n + sum(n-1)`

Questions:

- What does each stack frame return?
- When does addition actually happen?

☐ *Foundation of DP.*

Problem 6.4 – Power Function

Task:

Compute a^b using recursion.

✗ No loops

✗ No `Math.pow`

Question:

Why is this better than looping conceptually?

C. Stack Frame Awareness (CRITICAL)

Problem 6.5 – Trace the Stack (MANDATORY)

Task:

For `factorial(3)`, write:

- function calls
- return values
- order of execution

If you cannot write this manually → stop recursion here.

☐ *DFS and tree recursion rely on this exact understanding.*

Problem 6.6 – Print Before vs After

Task:

```
void fun(int n) {  
    if (n == 0) return;  
    System.out.print(n + " ");  
    fun(n - 1);  
    System.out.print(n + " ");  
}
```

Predict output for `fun(3)` WITHOUT running.

If you guess → you don't understand recursion.

D. Common Beginner Mistakes (YOU MUST FACE THESE)

Problem 6.7 – Infinite Recursion

Task:

Write a recursive function but forget to move toward base case.

Question:

Why does `StackOverflowError` occur?

☐ *Interview favorite.*

Problem 6.8 – Wrong Base Case

Task:

Write factorial with base case `if (n == 1)`.

Call `factorial(0)`.

Question:

Why does it crash?

☐ *Edge cases matter in DSA.*

E. Arrays + Recursion (DSA BRIDGE)

Problem 6.9 – Array Print

Task:

Print array elements using recursion.

Rules:

- Use index
- No loops

☐ *Used in tree traversals.*

Problem 6.10 – Find Max Element

Task:

Find maximum element in array using recursion.

Question:

What does each recursive call represent?

F. Thinking Like DSA (VERY IMPORTANT)

Problem 6.11 – Recursive Decision

Task:

Check if an array is sorted using recursion.

Logic required:

- Compare current element
- Trust recursion for rest

☐ *This is EXACT DSA thinking.*

Problem 6.12 – String Length Without `.length()`

Task:

Find length of string using recursion.

Hint:

Reduce problem size by substring or index.

☐ *Shows problem reduction skill.*

☐ **FINAL REALITY CHECK (READ CAREFULLY)**

If you think:

- Recursion is “calling a function again” → **You are wrong**
- Base case is optional → **You are wrong**
- Recursion is slower so useless → **You are wrong**

Correct thinking:

- Recursion = **problem** → **smaller problem**
 - Stack frames = **hidden memory**
 - Trees, graphs, DFS, DP = **pure recursion logic**
-

DO NOT MOVE FORWARD UNTIL:

You can:

- Predict outputs
- Draw stack frames
- Explain *why* recursion stops
- Explain *what each call returns*